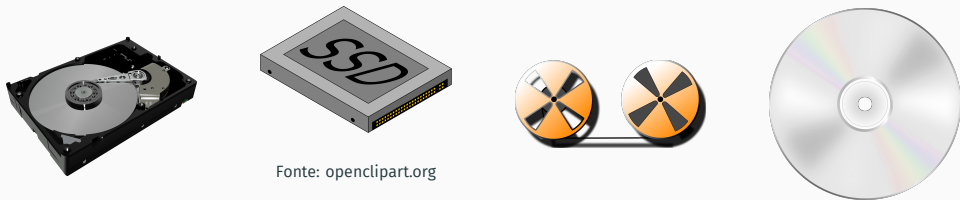


- todas as aplicações precisam **armazenar** e **recuperar** informações
- abordagem simples → usar seu espaço de endereçamento (RAM) durante a execução
 - desvantagens:
 1. pode ser pequeno demais
 2. volatilidade (i.e., informação é perdida quando o processo termina)
 3. compartilhamento entre mais de um processo é difícil
- precisamos de um **armazenamento de longo prazo**
 - requisitos:
 1. possibilidade p/ armazenar uma quantidade muito grande de informações
 2. informações devem sobreviver ao fim do processo que as está utilizando
 3. múltiplos processos têm de ser capazes de acessá-las ao mesmo tempo

- há diversos meios de armazenamento de longo prazo



- alguns tem partes móveis
- alguns oferecem acesso aleatório
 - outros apenas sequencial
- alguns tem desempenho ruim → usados apenas para *back-up*
- assuntos do Capítulo 5 (E/S)

- por enquanto vamos pensar no dispositivo de armazenamento como uma **sequência linear de blocos de tamanho fixo**
 - duas operações:
 1. ler o bloco k
 2. escrever o bloco k
 - resolvem o problema do armazenamento persistente
 - mas são pouco convenientes para sistemas maiores com múltiplos usuários (e.g., servidores)

- problemas recorrentes de um armazenamento persistente
 - *como encontrar informações?*
 - *como impedir que um usuário leia dados de outro?*
 - *como saber quais blocos estão livres?*
- o conceito de **abstração** foi útil anteriormente
 - Sistema Operacional abstrai o processador para criar o conceito de processo
 - cada processo vê um *processador virtual*
 - Sistema Operacional abstrai a memória física para oferecer *espaços de endereçamento* aos processos
 - cada processo pode acessar uma faixa de endereços (virtuais) de 0 até N
 - solução é adicionar uma nova abstração → **arquivos**

- um **arquivo** é uma unidade lógica de informação
 - criados|lidos|escritos por processos
 - um disco (potencialmente) conterá muitos arquivos
 - permitem modelar o disco
 - é como se cada arquivo fosse um conjunto de endereços (similar à memória)
 - arquivos são persistentes
 - só desaparece quando o proprietário o remove explicitamente
- o **sistema de arquivos** é a parte do SO que lida com arquivos

- estrutura
- nomeação
- acesso/uso

- proteção
- implementação
- gerenciamento



Fonte: openclipart.org

- inicialmente o foco será na interface
 - como um usuário pode usar um arquivo e quais as propriedades deste
 - ideia é que um arquivo é um mecanismo para armazenar informações no disco para depois poder lê-las
 - usuário não deve se preocupar com onde as informações estão ou como o disco de fato funciona

Nomeação de Arquivos

- i.e., como os objetos gerenciados são nomeados
- nome é definido na criação do arquivo por um processo
 - o arquivo continua existindo após o fim do processo criador
 - pode inclusive ser acessado por outros processos a partir do seu nome
- nomes são formados com *strings* (possivelmente incluindo caracteres especiais)
 - alguns sistemas diferenciam maiúsculas e minúsculas (outros não)
 - suarez vs Suarez vs SUAREZ

Nomeação de Arquivos

- muitos SOs trabalham com nomes de arquivos em duas partes separadas por . (ponto)
 - parte que segue o ponto é conhecida como extensão do arquivo
 - costuma indicar algo sobre o arquivo
 - e.g., MS-DOS possui nomes de 1 a 8 caracteres + extensão de até 3 caracteres
 - e.g., no UNIX qualquer decisão sobre extensão (ter ou não, mais de 1, etc) cabe ao usuário
 - além disso as extensões são apenas convenções (não são impostas pelo SO)
 - alguns programas (e.g., compilador) podem insistir que os arquivos tenha uma certa extensão
 - e.g., Windows é consciente das extensões e dá significado a elas
 - útil para definir um programa padrão para cada extensão

Nomeação de Arquivos

Extensão	Significado
.bak	Cópia de segurança
.c	Código-fonte de programa em C
.gif	Imagem no formato Graphical Interchange Format
.hlp	Arquivo de ajuda
.html	Documento em HTML
.jpg	Imagem codificada segundo padrões JPEG
.mp3	Música codificada no formato MPEG (camada 3)
.mpg	Filme codificado no padrão MPEG
.o	Arquivo objeto (gerado por compilador, ainda não ligado)
.pdf	Arquivo no formato PDF (Portable Document File)
.ps	Arquivo PostScript
.tex	Entrada para o programa de formatação TEX
.txt	Arquivo de texto
.zip	Arquivo compactado

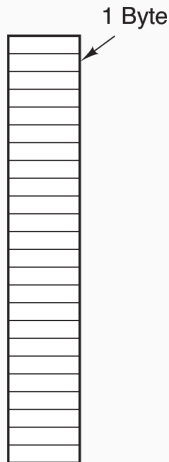


Estrutura de Arquivos

- arquivos podem ser estruturados de maneiras variadas

1. sequência desestruturada de *bytes*

- Sistema Operacional não sabe / não se importa sobre o que há no arquivo
 - qualquer significado deve ser imposto por programas em nível de usuário
- estratégia adotada pelo UNIX (incluindo Linux e OS X) e pelo Windows
- vantagem é a flexibilidade – programas podem colocar o que quiserem nos arquivos e nomeá-los da maneira mais conveniente
 - SO não ajuda mas também não atrapalha

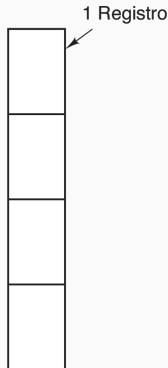


Estrutura de Arquivos

- arquivos podem ser estruturados de maneiras variadas

2. sequência de registros de tamanho fixo

- cada registro tem alguma estrutura interna
- operações ocorrem sobre 1 registro
 - i.e., lê 1 registro / escreve 1 registro
- e.g., sistemas antigos baseados em cartões perfurados de 80 colunas
 - cada arquivo consistia de registros de 80 caracteres

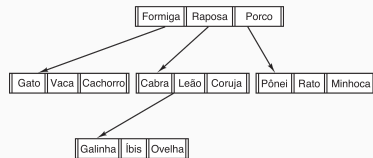


Estrutura de Arquivos

- arquivos podem ser estruturados de maneiras variadas

3. arquivo consiste numa árvore de registros

- os registros não são necessariamente do mesmo tamanho
- cada registro contém um campo chave
 - árvore é ordenada no campo chave → facilita a busca
- e.g., buscar pelo registro cuja chave é pônei (sem entrar em detalhes sobre a posição exata)
- ao adicionar um novo registro, o SO é quem decide onde colocá-lo
- é usado em alguns mainframes para processamento de dados comerciais



Tipos de Arquivos

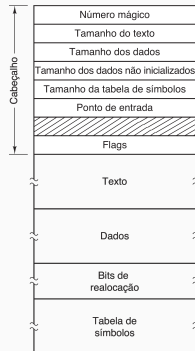
- é comum que os Sistemas Operacionais suportem vários tipos de arquivos
- UNIX (incluindo OS X) e o Windows oferecem
 - arquivos regulares
 - i.e., aqueles que contêm informações do usuário
 - diretórios
 - arquivos mantidos pelo sistema que guardam a estrutura do sistema de arquivos
- UNIX também oferece arquivos especiais
 - arquivos especiais de caracteres
 - relacionados com E/S; usados para modelar dispositivos seriais como terminais, impressoras e redes
 - arquivos especiais de blocos
 - usados para modelar discos
- neste momento vamos focar nos arquivos regulares!

Tipos de Arquivos

- arquivos regulares são arquivos ASCII ou arquivos binários
 - arquivos ASCII são formados por linhas de texto
 - podem ser exibidos e editados em qualquer editor de texto
 - fácil integração entre a saída de um programa e a entrada de outro
 - arquivos binários são aqueles que não são arquivos ASCII
 - possuem uma estrutura interna conhecida (“apenas”) pelos programas que os usam

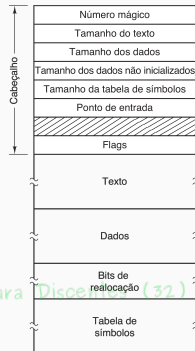
Tipos de Arquivos

- exemplo de um arquivo binário executável (de alguma versão antiga do UNIX)
 - todo SO precisa reconhecer pelo menos um tipo de arquivo → seu próprio executável



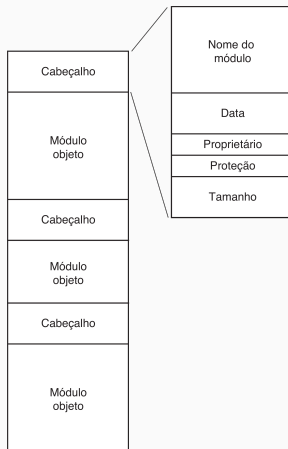
Tipos de Arquivos

- exemplo de um arquivo binário executável (de alguma versão antiga do UNIX)
 - todo SO precisa reconhecer pelo menos um tipo de arquivo → seu próprio executável
- tecnicamente é apenas uma sequência de bytes
 - porém o sistema vai interpretá-lo somente se tiver o formato apropriado
 - i.e., cabeçalho, texto, dados, *bits* de realocação e tabela de símbolos



Tipos de Arquivos

- exemplo de um arquivo binário repositório (archive) do UNIX
- guarda uma série de rotinas de biblioteca compiladas (mas não ligadas)
 - cada rotina é antecedida por um cabeçalho
 - nome
 - data de criação
 - proprietário
 - código de proteção
 - tamanho



Acesso aos Arquivos

- primeiros SOs ofereciam apenas **acesso sequencial**
 - i.e., processo pode ler todos os dados do arquivo em ordem
 - começa do início mas não pode pular nenhum registro
 - mas se necessário pode recomençar a leitura desde o início
 - era conveniente quando o meio de armazenamento era fita magnética
- com o surgimento dos discos passou a ser possível ler dados fora de ordem
 - e.g., acessar um registro usando sua chave e não sua posição
 - ao comprar uma passagem aérea, pode ir direto para o mapa de assentos do voo desejado (sem precisar ler os dados de milhares de outros voos antes)
 - i.e., agora temos **acesso aleatório**
 - operações precisam indicar onde começar a leitura
 - pode ser um parâmetro da chamada read
 - ou então uma combinação das chamadas seek e read (estratégia adotada pelo UNIX e pelo Windows)

Atributos de arquivos

- todo arquivo tem pelo menos
 - nome
 - data
- usualmente os Sistemas Operacionais adicionam outros detalhes
 - data/horario da última modificação
 - tamanho
 - atributos de proteção, *flags*, etc

(Exemplo de possíveis) Atributos de arquivos

Atributo	Significado
Proteção	Quem tem acesso ao arquivo e de que modo
Senha	Necessidade de senha para acesso ao arquivo
Criador	ID do criador do arquivo
Proprietário	Proprietário atual
Flag de somente leitura	0 para leitura/escrita; 1 para somente leitura
Flag de oculto	0 para normal; 1 para não exibir o arquivo
Flag de sistema	0 para arquivos normais; 1 para arquivos de sistema
Flag de arquivamento	0 para arquivos com backup; 1 para arquivos sem backup
Flag de ASCII / binário	0 para arquivos ASCII; 1 para arquivos binários
Flag de acesso aleatório	0 para acesso somente sequencial; 1 para acesso aleatório
Flag de temporário	0 para normal; 1 para apagar o arquivo ao sair do processo
Flag de travamento	0 para destravados; diferente de 0 para travados
Tamanho do registro	Número de bytes em um registro
Posição da chave	Posição da chave em cada registro
Tamanho da chave	Número de bytes na chave
Momento de criação	Data e hora de criação do arquivo
Momento do último acesso	Data e hora do último acesso do arquivo
Momento da última alteração	Data e hora da última modificação do arquivo
Tamanho atual	Número de bytes no arquivo
Tamanho máximo	Número máximo de bytes no arquivo

Operações com Arquivos

- operações para permitir armazenamento e recuperação de informações
 1. **Create** - cria um arquivo sem dados
 - serve pra anunciar que o arquivo está vindo e setar atributos
 2. **Delete** - remove o arquivo quando ele não é mais necessário
 3. **Open** - necessária antes de usar um arquivo
 - sistema busca os atributos e a lista de endereços do disco para a memória principal
 4. **Close** - quando os acessos são concluídos, remove os atributos e endereços liberando espaço nas tabelas internas
 - o arquivo é escrito em blocos, o fechamento força a escrita do último bloco mesmo que ainda não esteja cheio
 5. **Read** - lê dados do arquivo
 - em geral, lê a partir da posição atual
 - a quantidade de dados necessária e o *buffer* para salvá-los são argumentos da chamada

Operações com Arquivos

- operações para permitir armazenamento e recuperação de informações
- 6. **Write** - escreve dados no arquivo
 - normalmente na posição atual
 - se a posição atual for no fim do arquivo → aumentará de tamanho
 - se estiver no meio → (parte) dos dados será sobrescrita (i.e., perdida)
- 7. **Append** - uma variação de **Write** para acrescentar dados no final do arquivo
- 8. **Seek** - reposiciona o ponteiro de arquivo para que os dados possam ser lidos/escritos a partir desta posição
 - útil para especificar de onde tirar os dados em arquivos de acesso aleatório
- 9. **Get attributes** - permite que um processo recupere os atributos de um arquivo
 - e.g., `make` do UNIX examina a última alteração do arquivo fonte, se for mais recente que a última modificação do arquivo objeto, precisa recompilar

Operações com Arquivos

- operações para permitir armazenamento e recuperação de informações
 10. **Set attributes** - permite que o usuário altere atributos após a criação do arquivo
 - e.g., alterar o modo de proteção
 11. **Rename** - serve para mudar o nome de um arquivo
 - pode ser substituída pela cópia do conteúdo de um arquivo para outro com o novo nome seguida da remoção do arquivo antigo

Exemplo de programa para copiar um arquivo para outro

```
1  /* Programa que copia arquivos. Verific./relato de erros eh minima. */
2  #include <sys/types.h>
3  #include <fcntl.h>
4  #include <stdlib.h>
5  #include <unistd.h>
6
7  #define BUF_SIZE 4096      /* usa um tamanho de buffer de 4096 bytes */
8  #define OUTPUT_MODE 0700 /* bits de protecao p/ o arquivo de saida */
9
10 int main(int argc, char *argv[]) {
11
12     int in_fd, out_fd, rd_count, wt_count;
13     char buffer[BUF_SIZE];
14
15     if (argc != 3)
16         exit(1); /* erro de sintaxe se argc nao for 3 */
```

Exemplo de programa para copiar um arquivo para outro

```
17  /* Abre o arquivo de entrada e cria o arquivo de saída */
18  in_fd = open(argv[1], O_RDONLY); /* abre o arquivo de origem */
19  if (in_fd < 0)
20      exit(2); /* se não puder ser aberto, saia */
21
22  out_fd = creat(argv[2], OUTPUT_MODE); /* cria o arq. de destino */
23  if (out_fd < 0)
24      exit(3); /* se não puder ser criado, saia */
25
26  /* Laco de copia */
27  while (TRUE) {
28      rd_count = read(in_fd, buffer, BUF_SIZE); /* le 1 bloco de dados */
29      if (rd_count <= 0)
30          break; /* se fim de arquivo ou erro, sai do laco */
```

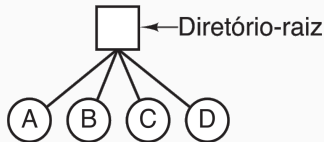
Exemplo de programa para copiar um arquivo para outro

```
31     wt_count = write(out_fd, buffer, rd_count); /* escreve dados */
32     if (wt_count <= 0)
33         exit(4); /* wt count <= 0 e um erro */
34 }
35
36 /* Fecha os arquivos */
37 close(in_fd);
38 close(out_fd);
39 if (rd_count == 0) /* nenhum erro na ultima leitura */
40     exit(0);
41 else
42     exit(5); /* erro na ultima leitura */
43 }
```

- além dos arquivos regulares, a maioria dos Sistemas Operacionais suporta o conceito de **diretórios** ou **pastas**
 - servem para organizar arquivos (e outros diretórios)

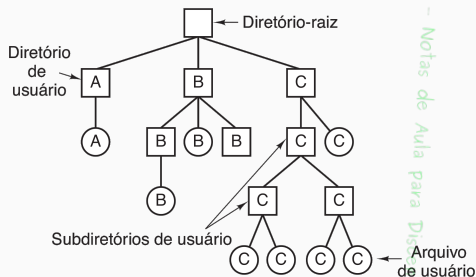
Sistemas de diretório em nível único

- forma mais simples é ter apenas um diretório que abriga todos os arquivos
 - as vezes conhecido como **diretório-raiz**
- estratégia adotada pelos primeiros computadores pessoais (só havia um usuário)
 - alguns dispositivos embarcados muito simples ainda usam esta solução
 - e.g., câmeras, *players*, etc
- vantagens
 - simplicidade
 - facilidade pra encontrar arquivos rapidamente
 - já que só há um lugar pra buscar



Sistemas de diretório hierárquicos

- com milhares de arquivos no mesmo diretório se torna muito difícil encontrar algo específico
 - é preciso uma maneira para agrupar arquivos em um mesmo local → hierarquia
 - i.e., uma árvore de diretórios
- um usuário pode ter tantos diretórios quanto julgar necessário
 - e.g., fotos, documentos, código, jogos, etc em um PC
- e.g., servidores que suportam vários usuários, cada um com o seu diretório-raiz privado



Nomes de caminhos

- a estrutura em árvore implica na necessidade de uma maneira de expressar os nomes dos arquivos
 - afinal, podemos ter dois arquivos diferentes com o mesmo nome em dois diretórios distintos
- **nome de caminho absoluto** é aquele que parte do diretório-raiz até o arquivo
 - e.g., `/usr/ast/caixapostal` (i.e., diretório-raiz que contém o diretório `usr` que contém o diretório `ast` que contém o arquivo `caixapostal`)
 - estes nomes sempre começam na raiz (`/`) e são únicos
 - UNIX usa `/` como separador, Windows `\` e MULTICS usava `>`
 - se o 1º caractere é o separador, então temos um nome de caminho absoluto

Nomes de caminhos

- **nome de caminho relativo** é usado em conjunto com o conceito de **diretório de trabalho** (ou atual)
 - usuário designa um diretório como sendo o de trabalho
 - todos os nomes de caminho que não comecem na raiz são interpretados como relativos ao diretório de trabalho
 - e.g., se o diretório de trabalho atual é `/usr/ast` o arquivo cujo caminho absoluto é `/usr/ast/caixapostal` pode ser acessado diretamente como `caixapostal`
 - os dois comandos abaixo farão exatamente a mesma coisa

```
cp /usr/ast/caixapostal /usr/ast/caixapostal.bak
cp caixapostal caixapostal.bak
```

Nomes de caminhos

- os programas podem usar nomes de caminho absoluto quando não quiserem levar em conta o diretório de trabalho atual
 - mas quando é preciso acessar muitos arquivos que residem no mesmo diretório, é mais fácil mudar o diretório e usar nomes relativos
- cada processo tem o próprio diretório de trabalho
 - quando ele é alterado, não interfere nos demais processos
 - nem no sistema de arquivos
 - i.e., mudar de diretório de trabalho sempre é seguro
- obs: se uma rotina de biblioteca muda o diretório de trabalho e não volta ao local de origem antes de terminar → programa inteiro pode quebrar

Nomes de caminhos

- a maioria dos sistemas com diretórios hierárquicos tem duas entradas especiais:
 - `.` (ponto) faz referência ao diretório atual
 - `..` (pontoponto) faz referência ao diretório pai
 - exceção: na raiz ele se refere a si mesmo
 - e.g., uso do comando `..` para subir na árvore
- e.g., três maneiras de fazer a mesma cópia (se o diretório de trabalho é `/usr/ast`)

```
cp ../lib/dictionary .
```

```
cp /usr/lib/dictionary .
```

```
cp /usr/lib/dictionary dictionary
```

```
cp /usr/lib/dictionary /usr/ast/dictionary
```

Operações com diretórios

- chamadas variam mais entre diferentes sistemas do que aquelas relacionadas à arquivos
- (alguns) exemplos do UNIX
 1. **Create** - cria um diretório vazio
 - contém apenas `.` e `..` colocados pelo sistema automaticamente
 2. **Delete** - remove um diretório (vazio)
 - um diretório vazio é aquele que contém apenas `.` e `..` (que não podem ser removidos)
 3. **Opendir** - abre um diretório (para que possa ser lido posteriormente)
 - e.g., abrir o diretório para então listar todos os arquivos ali contidos
 4. **Closedir** - fecha um diretório (libera espaço nas tabelas internas)
 5. **Readdir** - retornar a próxima entrada em um diretório aberto
 6. **Rename** - renomeia um diretório

Operações com diretórios

- (alguns) exemplos do UNIX

7. **Link** - cria uma ligação

- i.e., uma técnica p/ que um arquivo possa aparecer em mais de um diretório
- e.g., ligação estrita (*hard link*) que incrementa o contador no i-node do arquivo (monitorando o nº de entradas de diretório que contém o arquivo)

8. **Unlink** remove uma ligação (entrada de diretório)

- se o arquivo sendo removido estiver apenas em um diretório (caso usual), ele é removido
- caso esteja em mais de uma, apenas o nome de caminho em questão é removido
- no UNIX, a chamada de remover arquivos é implementada com `unlink`

- **ligação simbólica** (*alias*) é uma variação da ideia de ligação de arquivos

- é criado um nome que aponta para um arquivo minúsculo que nomeia outro
→ sistema de arquivos segue o caminho para encontrar o nome no final
- vantagem é que pode ser usada em discos diferentes (até mesmo remotos)
- desvantagem é que é menos eficiente